git-playbook.md

```
user@vscode:~/projeto$ ./domine-o-git.sh
```

Domine o Git no Terminal do VS Code

Um Playbook Visual de Sobrevivência para o Dia a a Dia (Focado em Comandos e Contexto)

Gemini Notebook



Git (O Motor Local)

// Sistema de controle de versão distribuído.

// Roda localmente na sua máquina.

// Focado em histórico, branches e merges (via CLI).

// É o coração da operação.



GitHub (A Plataforma Nuvem)

// Plataforma de hospedagem na web (Microsoft).

// Facilita colaboração e Pull Requests.

// Revisão de código e gestão de times.

Git é a ferramenta (o que você digita no VS Code). **GitHub** é o destino (para onde o código vai).

Gemini Notebook

Os 4 Estágios do Código

Working Directory
(Diretório de Trabalho)

Sua máquina. Arquivos modificados que o Git ainda não está rastreando.

Staging Area
(Área de Preparação)

A zona de embarque. Arquivos selecionados e prontos para serem salvos.

Local Repository
(Repositório Local)

Seu histórico seguro no PC. Os snapshots confirmados.

Remote Repository
(Repositório Remoto)

A nuvem (GitHub). Código compartilhado com o time.

Gemini Notebook

Iniciando os Trabalhos

Cenário: O projeto já existe no GitHub

```
git clone [url_do_repositorio]
```



Traz o repositório inteiro (com todo o histórico) da Nuvem para o seu Computador.

Cenário: Estou começando do zero absoluto

```
git init
```



Transforma uma pasta comum em um repositório Git local. Cria a pasta oculta **.git**.

```
// Se usar git init, você precisará conectar ao GitHub depois usando: git remote add origin [url]
```

Gemini Notebook

O Ciclo de Salvamento



1. Preparando (Staging)

```
git add .
```



Você diz ao Git: Preste atenção nestas alterações para o meu próximo salvamento.

2. Salvando (Commit)

```
git commit -m "feat: adiciona botão de login"
```



Tira uma fotografia permanente das alterações.

Boa Prática: Use títulos curtos (menos de 50 caracteres) e descritivos. Diga o que o commit faz.

Gemini Notebook

Onde eu estou? (Lendo os Radares)

```
user@vscode:~$ git status
On branch feature-login
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   src/app.js

Changes not staged for commit:
        modified:   styles.css
```

O GPS imediato. Use sempre que voltar do café.

```
user@vscode:~$ git log --oneline
a1b2c3d (HEAD -> feature-login) feat: adiciona botão de login
f9e8d7c fix: corrige padding do header
b3a2c1d (origin/main, main) docs: atualiza readme
```

A Máquina do Tempo. Mostra o histórico condensado.

Gemini Notebook

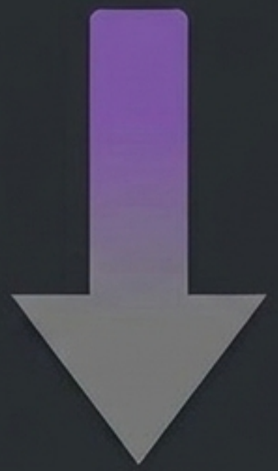
Sincronizando com a Nuvem

```
git push origin [nome-da-branch]
```



Enviando.
Transmite os seus commits locais para o GitHub.
Use quando terminar uma funcionalidade.

```
git pull
```



Recebendo.
Busca as novidades do GitHub e mescla (merge) automaticamente com o seu código.
Obrigatório antes de começar a codar pela manhã.

Gemini Notebook

Por que usar Branches? (Universos Paralelos)



O Problema:
Trabalhar direto na main causa quebras de código e conflitos com outros desenvolvedores.

A Solução:
Ramos isolados. Uma branch é um ambiente seguro onde você pode testar, errar e construir sem afetar o código de produção que os clientes usam.

```
// No Git, criar branches é quase instantâneo e não consome espaço extra.
```

Gemini Notebook

Comandos de Branching (Cheat Sheet)

Listar Branches

```
git branch
```

Mostra suas branches locais. A ativa fica com um asterisco verde *.

Criar e Mudar de Contexto (O Mais Usado)

```
git checkout -b [branch]
git switch -c [branch]
```

Cria um novo universo paralelo e teleporta você para ele em um único comando.

Apenas Mudar

```
git checkout [branch]
git switch [branch]
```

Muda para uma branch que já existe.

Limpeza

```
git branch -d [branch]
```

Apaga branches antigas após terem sido fundidas (merged).

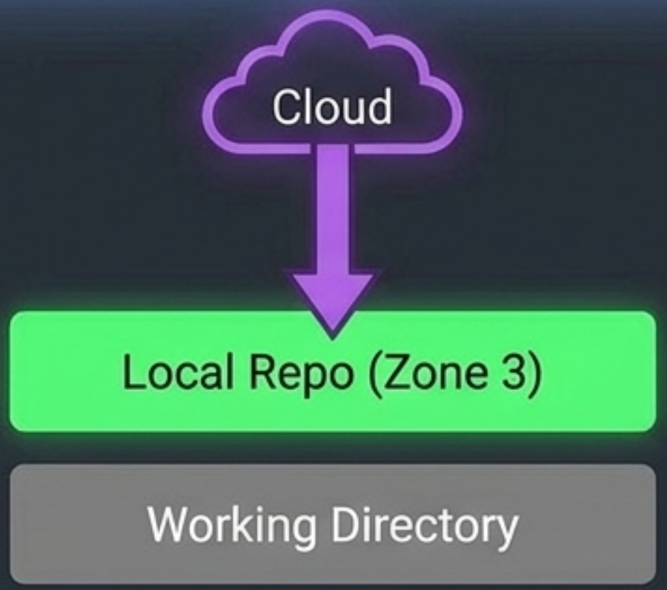
Gemini Notebook

Trazendo atualizações: A nuance segura

Seu código está desatualizado, mas você não quer misturar os arquivos automaticamente.

A Abordagem Segura

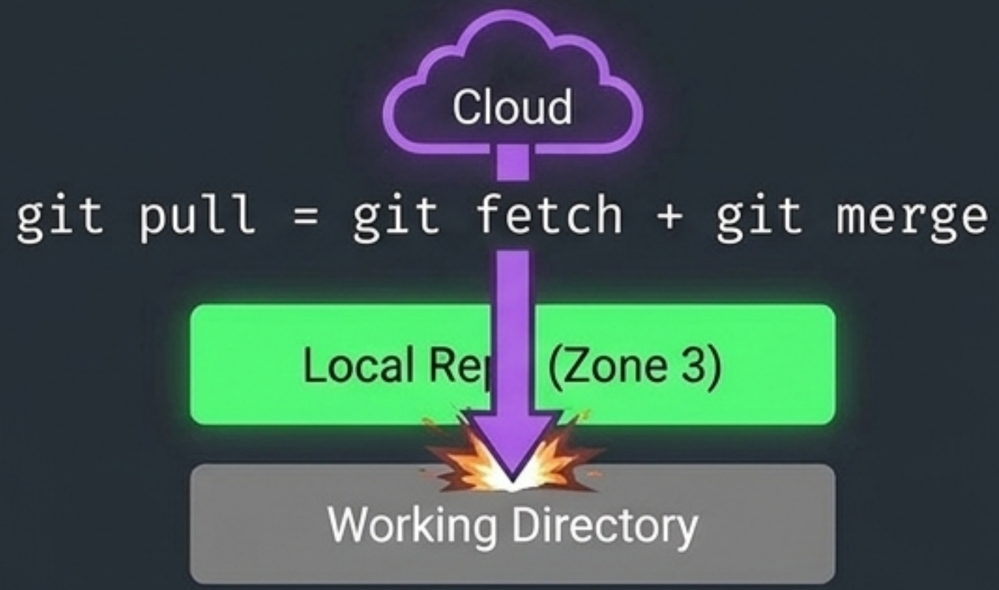
```
git fetch
```



Funciona como o botão Atualizar da caixa de e-mail. Baixa as novidades da nuvem, mas NÃO as aplica nos seus arquivos abertos no VS Code.

O Atalho

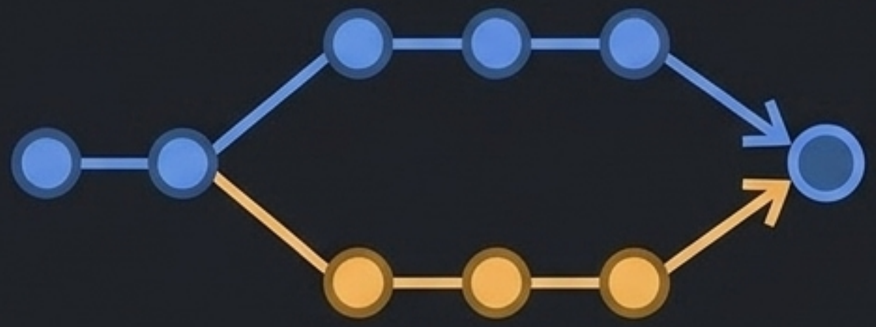
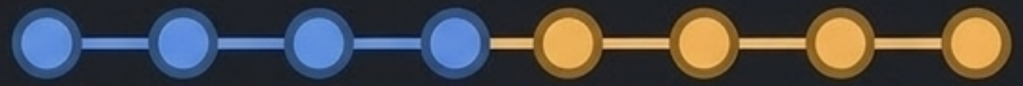
```
git pull
```



Faz o download e força a mistura instantânea nos seus arquivos.

Gemini Notebook

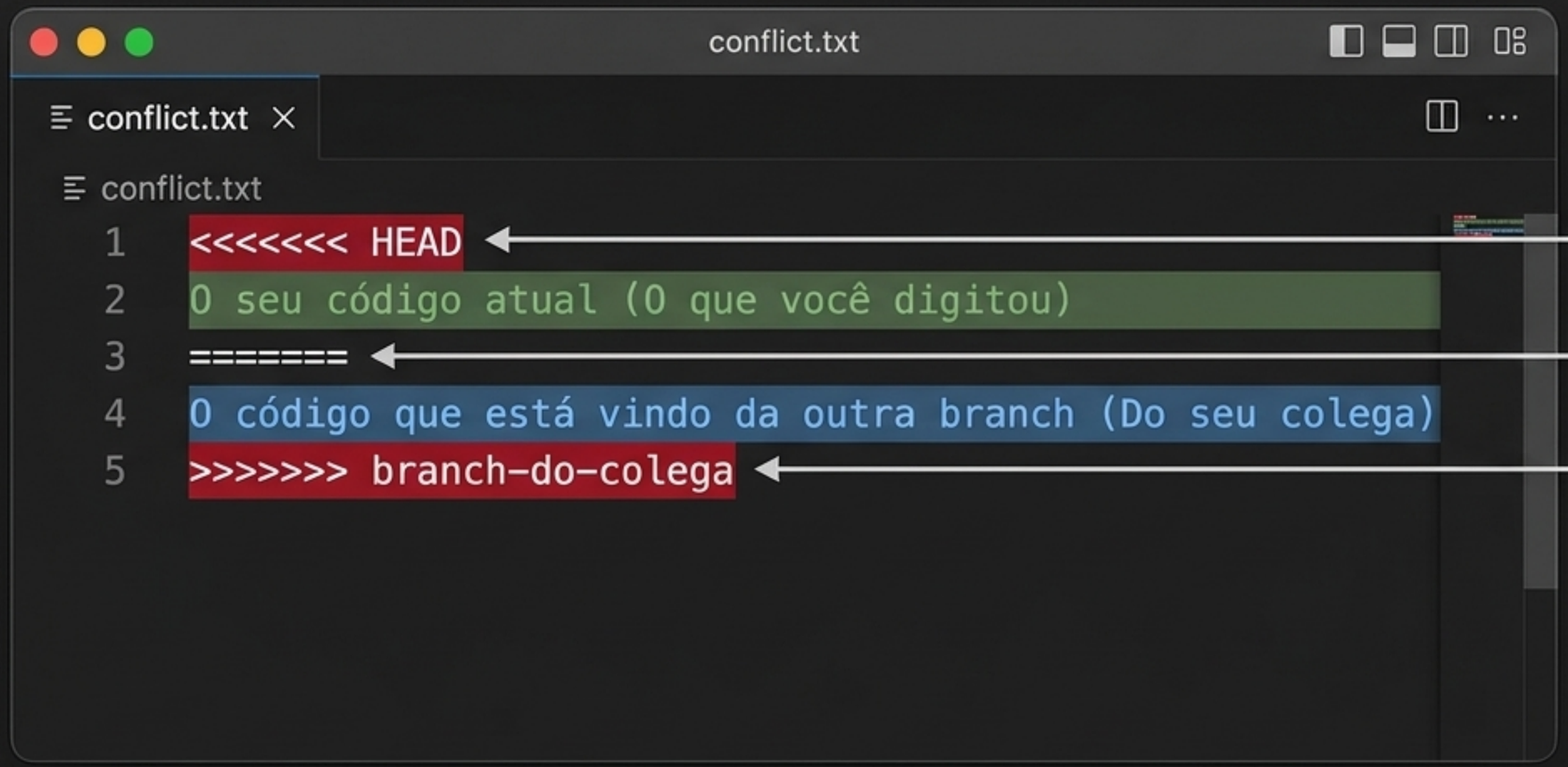
A Convergência: Merge vs. Rebase

`git merge main`	`git rebase main`
	
✅ Preserva o histórico exato. ✅ Estrutura não-linear. ✅ Gera um merge commit extra.	✅ Reescreve o histórico de commits. ✅ Estrutura 100% limpa e linear. ✅ Evita commits de merge desnecessários.
Veredito Mais seguro. Ideal para times grandes e branches compartilhadas.	**Veredito** Avançado. Ideal para histórico limpo, mas NUNCA use em branches já compartilhadas publicamente (gera caos).

Gemini Notebook

Zona de Conflito: A Anatomia

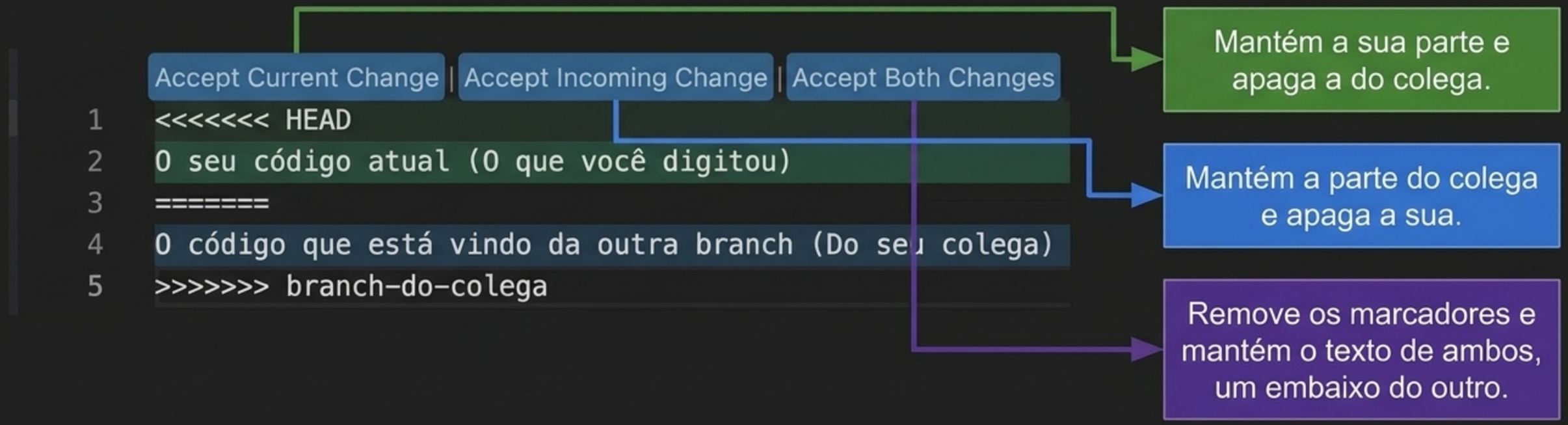
Por que ocorre? Duas pessoas editaram a mesma linha em branches diferentes.



Gemini Notebook

Lendo a Interface Gráfica (VS Code Lens)

O VS Code traduz os marcadores do terminal em botões clicáveis.



Nota: Você **não precisa usar a interface gráfica**. Deletar os marcadores e ajustar o texto manualmente no teclado tem exatamente o mesmo efeito.

Gemini Notebook

Resolvendo o Conflito na Prática



1 Edição

Resolva o texto do arquivo. Use os botões do VS Code ou digite.
O arquivo final deve ficar limpo, SEM os marcadores (<<<, ===, >>>).



2 Staging

`git add [nome-do-arquivo]`

Isso diz ao Git: "O conflito neste arquivo foi curado".



3 Finalização

`git commit`

Atenção: Não use o -m. O Git abrirá um editor no terminal com uma mensagem de merge gerada automaticamente. Apenas feche e salve o arquivo para finalizar.

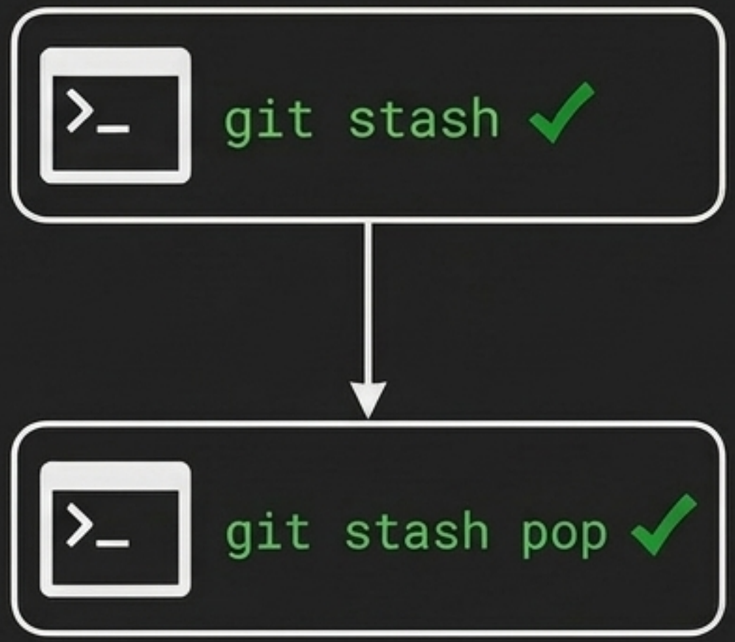
Sucesso: Sua branch agora contém as atualizações com o conflito resolvido.

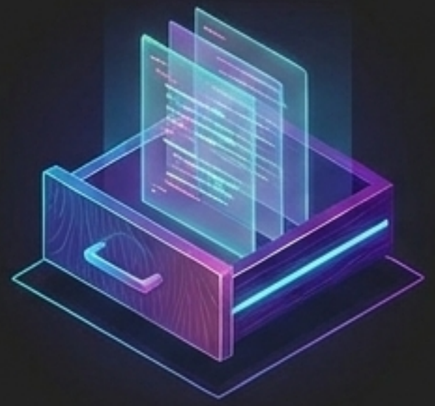
Gemini Notebook

O Botão de Interrupção (Stash)

A Dor

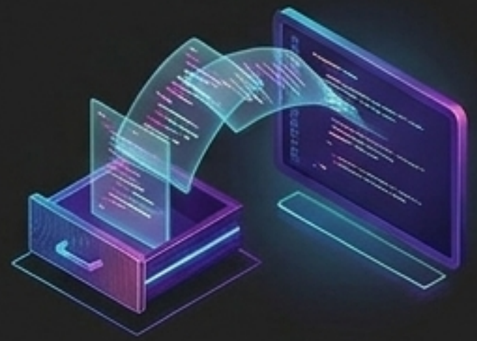
Cenário: Você está no meio de uma feature quebrada e o chefe pede uma correção urgente na main. Você não pode comitar código pela metade.





A Solução Mágica

O Git pega suas alterações não-comitadas, joga em uma gaveta temporária e limpa sua tela. Mude para a main livremente.



A Retomada

Volte para sua branch. O código da gaveta é despejado de volta exatamente como você o deixou.

Gemini Notebook

O Botão de Pânico (Desfazendo Besteiras)

Fiz besteira no meu último commit. Como eu volto no tempo?

O Caminho Seguro

```
git reset --soft HEAD~1
```

O último commit é desfeito, MAS os arquivos modificados voltam para a sua Staging Area.

Você não perde o código digitado, apenas desfaz o agrupamento do commit.

A Opção Nuclear

```
git reset --hard HEAD~1
```

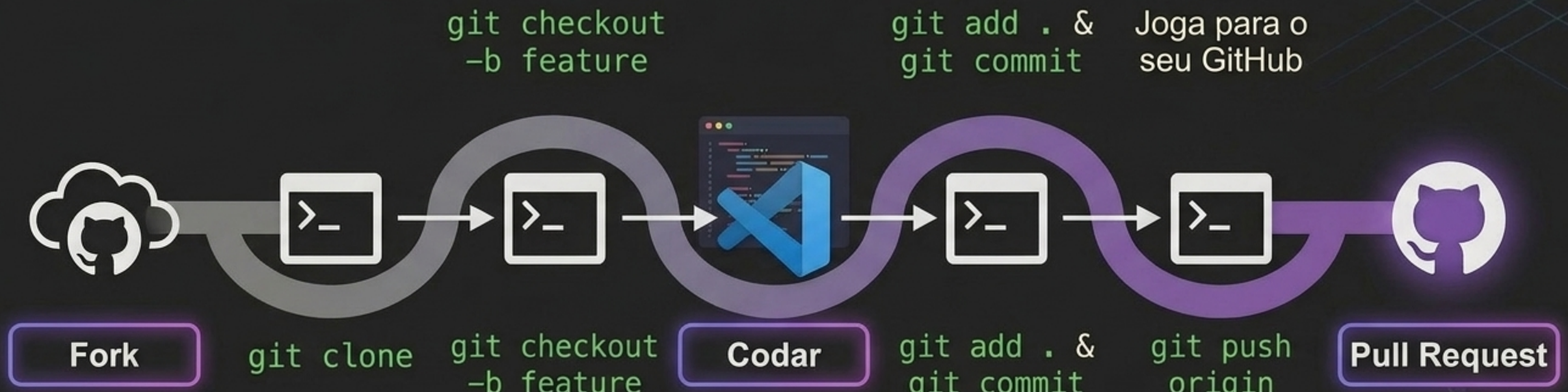
O último commit é destruído E todas as alterações nos arquivos são apagadas para sempre.

Seu código volta a ser exatamente o que era antes. Cuidado extremo.

Gemini Notebook

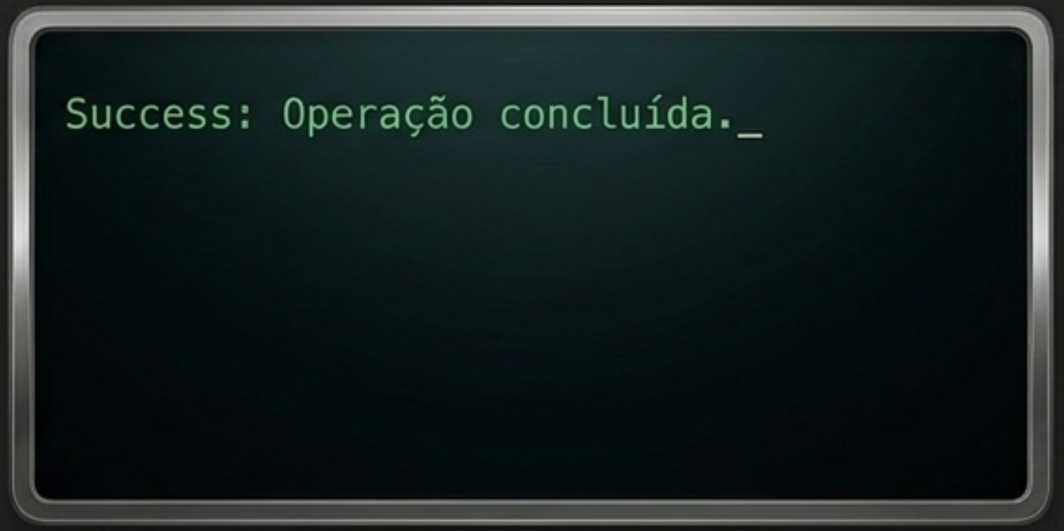
A Síntese: O Ciclo de Vida do Pull Request

O fluxo de contribuição Open Source unindo o Terminal ao GitHub.



Gemini Notebook

Dica de Ouro no VS Code



O VS Code possui uma excelente aba Source Control integrada para cliques rápidos.

MAS...

Entender os comandos por baixo dos panos e ler as respostas do terminal dá a você total controle, capacidade de diagnóstico e resolve a síndrome do 'cliquei no botão e não sei o que aconteceu'.

A interface é uma conveniência. O terminal é a fundação. Mantenha os comandos nos dedos.

Gemini Notebook